Assignment 1 Report

Namma Metro Smart-Card Fare System

Design, Implement and Validate the Fare Logic

Student Name:	NEELAKANDA MOORTHY
Register Number:	192421063
Course:	CO1 - Object-Oriented Programming in C++
Bloom's Level:	K5 - Evaluate
Compiler:	MinGW-W64 GCC 8.1.0
Compile Command:	g++ -std=c++17 -Wall -Wextra
GitHub Repository:	github.com/moorthy-dnkkw/DSA0119-C-

Table of Contents

Task 1 - Understand the Problem and Identify the Objects (4 Marks)

Task 2 - Use Tokens, Keywords, Identifiers and Constants (3 Marks)

Task 3 - Choose Data Types and Check Type Compatibility (4 Marks)

Task 4 - Control Implicit Conversions (4 Marks)

Task 5 - Write Correct Expressions with Right Precedence (4 Marks)

Task 6 - Design the Control Structures (4 Marks)

Task 7 - From Procedural to Object-Oriented (4 Marks)

Task 8 - Read and Print Data Safely (4 Marks)

Task 9 - Implement and Test the Fare Logic (4 Marks)

Task 10 - Compare Expectations with Execution (4 Marks)

Task 11 - Recommend and Defend the Final Design (4 Marks)

Task 12 - Day-4 Design Challenge (8 Marks)

Task 1 - Understand the Problem and Identify the Objects (4 Marks)

Objects, Data, and Behaviour

Object	Data (Attributes)	Behaviour (Methods)	Justification
SmartCard	cardId, balancePaise	canTravel(), deduct()	Core state holder
Journey	zone, hour, fare	execute(), printSummary()	Encapsulates trip
FareTable	BASE_PAISE[] array	isPeak(), calculateFare()	Single fare source
Passenger	name, age, type	getConcession()	Discount logic
Zone	Inner, Middle, Outer	N/A (enum class)	Type-safe values

OOP Principles in the Metro System

Principle	Application in Metro System
Encapsulation	SmartCard keeps balancePaise private; only deduct()/recharge() can modify it
Abstraction	FareTable exposes calculateFare() hiding surcharge formula internals
Inheritance	Student/SeniorCitizen types can inherit from base Passenger class
Polymorphism	Different passenger types override getConcession() for different discounts

Invariant Rules

- Balance after any journey must remain >= Rs 50.00 (5000 paise)
- Zone values must be within valid range (Inner/Middle/Outer)
- Recharge amount must be positive (> 0)
- Peak hours: [8, 10) and [17, 20) -- 10:00 is off-peak, 20:00 is off-peak

Alternative Concept: Procedural vs OOP

Aspect	Procedural	Object-Oriented	Winner
Time/journey	O(1)	O(1)	Tie
Memory/card	O(1)	O(1)	Tie
Min-bal change	Every if check	1 line in class	OOP
Corruption risk	HIGH (globals)	ZERO (private)	OOP

Task 2 - Tokens, Keywords, Identifiers and Constants (3 Marks)

Five Token Categories with Examples

Token Type	Examples from Code
Keywords	enum, class, constexpr, long, bool, switch, case, return, const, static
Identifiers	Zone, FareConfig, INNER_ZONE_PAISE, calculateFare, SmartCard
Constants	1000LL, 20, 5000LL, true, "Inner"
Operators	::, =, +, *, /, ==, >=, &&, ||, <<
Punctuators	{, }, (,), ;, :, comma, .

Constant Mechanism Comparison

Feature	#define	const	constexpr	enum	enum class
Type Check	None	Yes	Yes (strict)	Weak	Strong
Scope	File	Block	Block	Enclosing	Scoped
Eval Time	Preproc	Runtime	Compile	Compile	Compile
Memory	0 bytes	May alloc	0 bytes	0 bytes	0 bytes
Debugger	Invisible	Visible	Visible	Visible	Visible

Verdict: constexpr is preferred -- it provides type safety, scope control, compile-time evaluation, and zero runtime overhead. enum class is used for Zone values for type-safe scoped enumeration without implicit int conversion.

Task 3 - Choose Data Types and Check Type Compatibility (4 Marks)

Data Type Selection Table

Value	Type	sizeof	Justification
Fares (paise)	long long	8 bytes	Network-scale accumulation exceeds INT_MAX
Balances	long long	8 bytes	Same reasoning as fares
Hour of travel	int	4 bytes	Range 0-23, well within int range
Zone number	enum class	4 bytes	Type-safe scoped enumeration
Journey count	long long	8 bytes	Up to 3,000,000 per day
Peak flag	bool	1 byte	Binary true/false

Why Integer Paise, Not Double Rupees

- double cannot exactly represent Rs 10.10 in binary (IEEE 754)
- Over 100,000 additions of Rs 10.10: double drifts by ~0.00016 Rupees
- long long integer paise: EXACT arithmetic, zero cumulative error

Network Scale Overflow Proof

3,000,000 journeys x Rs 25 avg = Rs 75,000,000 = 7,500,000,000 paise

INT_MAX = 2,147,483,647 paise --> OVERFLOW by 5.35 billion paise!

LLONG_MAX = 9.2 x 10^18 --> Handles years of network data safely.

Task 4 - Control Implicit Conversions (4 Marks)

Predicted Output (Written Before Running)

```
3 3.5
C 68
1 true
s >= u
```

Actual Output (Matches Prediction)

```
3 3.5
C 68
1 true
s >= u
```

Line-by-Line Conversion Analysis

Line	Code	Conversion	Explanation
1	a / b -> c	Int div, int->double	7/2=3 (truncated), widened to 3.0
1	static_cast(a)/b	Explicit cast	7.0/2.0 = 3.5 (exact)
2	'A' + 2 -> ch	Promotion, narrow	65+2=67 (C), narrowed to char
2	ch + 1	Promotion char->int	'C'(67)+1=68 printed as integer
3	bool flag = 5	int->bool	5 != 0, so flag = true (1)
4	s < u	signed->unsigned	-1 becomes 4294967295u; FALSE

Prevention Techniques

- Brace initialization: int x{3.7}; --> compile-time ERROR (narrowing detected)
- static_cast: Makes intent explicit, aids code review
- Same-type comparisons: Avoid mixing signed and unsigned

Task 5 - Correct Expressions with Right Precedence (4 Marks)

Predicted Output (Written Before Running)

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Actual Output (Matches Prediction)

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Operator Breakdown

Expression	Operators	Precedence Rule	Result
x + y * 2	*(P5) > +(P6)	Multiply first	5 + 6 = 11
(x + y) * 2	Parens override	Addition first	8 x 2 = 16
x++ + ++y	Post(P2) Pre(P3)	x=5 then 6, y=4	5 + 4 = 9
x % 4 * 2	% * same(P5) LR	(6%4)*2	2 x 2 = 4
-10 / 3	Truncate to 0	C++11 rule	-3
-10 % 3	a==(a/b)*b+a%b	Identity	-1

Task 6 - Design the Control Structures (4 Marks)

SIMATS Pseudocode

```
ALGORITHM CalculateFare(zone, isPeak)
INPUT:  zone of type Zone, isPeak of type Boolean
OUTPUT: finalFare in integer paise
BEGIN
  SWITCH zone DO
    CASE Zone.Inner:  baseFare <- 1000
    CASE Zone.Middle: baseFare <- 2000
    CASE Zone.Outer:  baseFare <- 3000
  END SWITCH
  IF isPeak THEN
    finalFare <- baseFare + (baseFare*20)/100
  ELSE
    finalFare <- baseFare
  RETURN finalFare
END
```

Completed Code Output (Matches Expected)

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Boundary Tests

- 10:00 AM: OFF-PEAK (standard Rs 30 fare) - Verified
- 8:00 PM (20:00): OFF-PEAK (standard Rs 30 fare) - Verified

Alternative: Array Lookup vs Switch

Array: FARE[static_cast<int>(z)] -- O(1), direct memory read, trivial to extend.

Switch: O(1) (compiler may optimize to jump table), requires new case for each zone.

For extensibility, array lookup is superior.

Task 7 - From Procedural to Object-Oriented (4 Marks)

Side-by-Side Comparison

Aspect	Procedural	Object-Oriented	Winner
Balance storage	Global long long	Private member	OOP
Invariant	None (any fn)	Enforced by deduct()	OOP
Min-bal change	Every if check	1 line constant	OOP
Time complexity	O(1)	O(1)	Tie
Space complexity	O(1)	O(1)	Tie

Key Finding

OOP achieves complete state protection at zero runtime cost. The procedural version was demonstrated to allow direct corruption (globalCardBalance = -2500), while the OOP version prevents this at compile time via private access.

Task 8 - Read and Print Data Safely (4 Marks)

Stream Error Recovery Mechanism

- cin >> value fails --> cin.fail() returns true
- cin.clear() clears the error flags
- cin.ignore(numeric_limits<streamsize>::max(), newline) discards the corrupt buffer
- Loop re-prompts the user

Formatted Output

- std::setw(N) -- field width
- std::setfill('0') -- zero-padding for paise
- std::fixed << std::setprecision(2) -- 2 decimal places for rupees

Alternative: getline + stoi vs cin >>

Aspect	cin >>	getline + stoi	Winner
Time	O(1) per number	O(L) per line	cin
Space	O(1)	Heap string	cin
Error handling	Failbit + clear	try/catch except	Tie

Task 9 - Implement and Test the Fare Logic (4 Marks)

Automated Test Results: 13/13 PASS

Test	Description	Expected	Actual	Status
TC-01	Inner Off-Peak	Rs 10.00	Rs 10.00	PASS
TC-02	Inner Peak	Rs 12.00	Rs 12.00	PASS
TC-03	Middle Off-Peak	Rs 20.00	Rs 20.00	PASS
TC-04	Middle Peak	Rs 24.00	Rs 24.00	PASS
TC-05	Outer Off-Peak	Rs 30.00	Rs 30.00	PASS
TC-06	Outer Peak	Rs 36.00	Rs 36.00	PASS
TC-07	08:00 Boundary (Peak)	Rs 36.00	Rs 36.00	PASS
TC-08	10:00 Boundary (Off-Peak)	Rs 30.00	Rs 30.00	PASS
TC-09	17:00 Boundary (Peak)	Rs 36.00	Rs 36.00	PASS
TC-10	20:00 Boundary (Off-Peak)	Rs 30.00	Rs 30.00	PASS
TC-11	Leaves exactly Rs 50	APPROVED	APPROVED	PASS
TC-12	Leaves Rs 49.99	DENIED	DENIED	PASS
TC-13	Invalid Zone	DENIED	DENIED	PASS

Hand-Calculated 5-Journey Trace

Total approved fares: Rs 12 + Rs 20 + Rs 10 = Rs 42.00 (4200 paise)

Program accumulated: 4200 paise -- EXACT MATCH

Task 10 - Compare Expectations with Execution (4 Marks)

Scale Simulation Results

Scale	long long	double (Rs)	Drift (Rs)	Overflow?
1 Station 20K	40,700,000	407,000.00	~0.000	No
1 Line 500K	1,017,500,000	10,175,000	~0.000096	No
Network 3M	6,105,000,000	61,050,000	~0.003	YES

Key Findings

- 32-bit int catastrophically overflows at network scale (6.1B > INT_MAX 2.1B)
- double accumulates Rs 0.003 drift over 3M journeys (unacceptable for audit)
- long long is exact -- zero error at any scale

Task 11 - Recommend and Defend the Final Design (4 Marks)

Recommended Production Architecture

Component	Choice	Justification
Zone	enum class : int	Type-safe, no implicit conversion
Fare Table	constexpr array	Single revision point, zero overhead
Fare Engine	Static pure funcs	Stateless, testable, O(1)
SmartCard	Private balance	Invariant: bal >= Rs 50 after deduction
Accumulator	long long (64-bit)	Prevents int overflow at 3M+ scale
Currency	Integer paise	Eliminates IEEE 754 rounding drift

Fare Revision Impact

To update Inner Zone from Rs 10 to Rs 15: change FARE_TABLE_PAISE[0] from 1000 to 1500. Total lines changed: 1.

Task 12 - Day-4 Design Challenge (8 Marks)

A. Impact Analysis (2 Marks)

Changed Requirements:

- Fourth zone: Airport (Rs 60 / 6000 paise) -- add 1 enum value + 1 array element
- Weekend discount: 10% off, does NOT combine with peak surcharge
- Monthly pass: Unlimited journeys in 2 chosen zones (0 fare)

Affected Code: Zone enum (1 line), fare array (1 line), FareEngine (10 lines), SmartCard (15 lines), new MonthlyPass class (20 lines).

B. Evaluation of Original Design (2 Marks)

- Switch-based fare logic easily absorbs 4th Airport case
- Weekend discount breaks simple "if (peak) surcharge" pattern -- needs restructuring
- Monthly pass requires SmartCard to bypass fare engine entirely -- needs extension

Verdict: MODIFY the original design. Retain SmartCard core, redesign FareEngine with priority-based discount logic, add new MonthlyPass class.

C. Adaptation Implementation (2 Marks)

- Array-indexed fare table replaces switch (O(1), extensible)
- Priority-based discount logic: Weekend discount > Peak surcharge > Standard fare
- MonthlyPass class with coversZone() check
- SmartCard.processJourney() checks pass FIRST, then calls FareEngine

D. Evidence-Based Defence: 16/16 Tests PASS (2 Marks)

Test	Scenario	Expected	Actual	Status
T01	Inner Weekday Off-Peak	Rs 10.00	Rs 10.00	PASS
T02	Inner Weekday Peak	Rs 12.00	Rs 12.00	PASS
T03	Middle Weekday Peak	Rs 24.00	Rs 24.00	PASS
T04	Outer Weekday Off-Peak	Rs 30.00	Rs 30.00	PASS
T05	Outer Weekday Peak	Rs 36.00	Rs 36.00	PASS
T06	Airport Weekday Off-Peak	Rs 60.00	Rs 60.00	PASS
T07	Airport Weekday Peak	Rs 72.00	Rs 72.00	PASS
T08	Inner Weekend Off-Peak	Rs 9.00	Rs 9.00	PASS
T09	Outer Weekend Peak Time	Rs 27.00	Rs 27.00	PASS
T10	Airport Weekend Peak	Rs 54.00	Rs 54.00	PASS
T11	Pass: Inner Covered	PASS-FREE	PASS-FREE	PASS
T12	Pass: Middle Covered	PASS-FREE	PASS-FREE	PASS
T13	Pass: Outer NOT Covered	Rs 30.00	Rs 30.00	PASS
T14	Pass: Airport NOT Covered	Rs 72.00	Rs 72.00	PASS
T15	Exact Rs 50 boundary	APPROVED	APPROVED	PASS
T16	Denied - low balance	DENIED	DENIED	PASS

Complexity Analysis

Time Complexity: O(1) per journey (unchanged from original 3-zone design)

Space Complexity: O(Z) for fare table + O(1) per card (Z = number of zones)

Adding a 5th zone requires changing exactly 2 lines: enum value + array element.